

Universidad de Costa Rica

Facultad de Ingeniería

Escuela de Ingeniería Eléctrica

IE0623 - Microprocesadores

II ciclo 2025

Proyecto Final
Velodromo 623

Danna Guevara Quesada C23562

Grupo 01

Profesor: Geovanny Delgado Cascante

Fecha de entrega: 5 diciembre

Índice

1. Diseño de la aplicación	2
1.1. Programa Principal	2
1.2. Tarea_Modo_Espera	3
1.3. Tarea_Configurar	3
1.4. Tarea_Modo_Correr	5
1.5. Tarea_Modo_Resumen	6
1.6. Tarea_Brillo	7
1.7. Tarea_Teclado	8
1.8. Tarea_Leer_Teclado	9
1.9. Tarea_Led_Testigo	10
1.10. Tarea_Leer_PB1	10
1.11. Tarea_Leer_PB2	11
1.12. Tarea_PantallaMUX	12
1.13. Tarea_LCD	13
1.14. Tarea_Send_LCD	14
1.15. Tarea_BCD_BIN	15
1.16. Subrutina_Calcula	16
1.17. Subrutina_BIN_BCD_MUXP	18
1.18. Subrutina_BCD_7SEG	19
1.19. Subrutina_Borrar_NumArray	20
1.20. Máquina de Tiempos	20
2. Estructuras de datos	22
3. Conclusiones	28
4. Recomendaciones	28
5. Bibliografía	28

Resumen

El proyecto Velódromo 623 consiste en el diseño, implementación y validación de un sistema embebido capaz de medir la velocidad del ciclista, contabilizar las vueltas recorridas y desplegar información de forma sincronizada durante un ciclo de entrenamiento. Para ello, se utilizaron dos sensores ubicados a una distancia fija entre sí, cuya interrupción permite medir el tiempo transcurrido entre los puntos S1 y S2. A partir de esta medición, el sistema calcula la velocidad instantánea del ciclista, genera los tiempos de despliegue y determina la información que debe mostrarse antes y después de que el ciclista alcance la pantalla. Todo el comportamiento del sistema se rige mediante una arquitectura modular basada en tareas independientes, cada una implementada como una máquina de estados que opera de manera concurrente bajo un despachador cíclico.

El diseño incorpora múltiples periféricos: una pantalla LCD de dos líneas para mensajes descriptivos, un display multiplexado de cuatro dígitos para información numérica, un teclado matricial de dieciséis teclas para interacción del usuario, LEDs indicadores de modo, un potenciómetro analógico para el control de brillo y un microrelé que actúa como alarma visual al finalizar el entrenamiento. Cada uno de estos componentes se integra mediante tareas dedicadas: la tarea de teclado interpreta secuencias numéricas y teclas funcionales; las tareas Leer PB1 y Leer PB2 se encargan del antirrebote y clasificación de pulsaciones asociadas a los sensores S1 y S2; la tarea Pantalla MUX administra el refrescamiento de los dígitos y modula su brillo utilizando la conversión analógica del potenciómetro; la tarea LCD controla el envío de cadenas completas al módulo de pantalla y coordina su operación con la tarea Send LCD, que transmite físicamente los nibbles de datos siguiendo las restricciones temporales del controlador.

El sistema implementa cuatro modos de operación: Espera, Configurar, Correr y Resumen. En el modo Espera se mantiene el sistema ocioso y se presenta el mensaje inicial. En el modo Configurar, el usuario define la cantidad de vueltas del entrenamiento mediante una máquina de estados que recibe la entrada del teclado, valida el valor y actualiza el parámetro NumVueltas. El modo Correr constituye el núcleo de la aplicación, gestionando la detección de sensores, el cálculo de velocidad, la verificación de rangos permitidos, el manejo de tiempos TimerIniPant y TimerFinPant, la acumulación de velocidades y el despliegue sincronizado de resultados en LCD y en el display de 7 segmentos. En este modo, la subrutina Calcula implementa las ecuaciones derivadas para obtener la velocidad en km/h y los tiempos para determinar cuándo debe aparecer y desaparecer la información en pantalla. El sistema también es capaz de reconocer velocidades fuera del rango permitido y desplegar un mensaje de alerta sin afectar el conteo total de vueltas. Finalmente, en el modo Resumen el sistema calcula la velocidad promedio del entrenamiento y presenta tanto esta como la cantidad de vueltas realizadas, concluyendo así el ciclo operativo.

La robustez temporal del sistema se garantiza mediante la Máquina de Tiempos, la cual decrementa periódicamente todas las tablas de temporizadores utilizados por las tareas. Asimismo, se emplean subrutinas adicionales para convertir valores entre formatos binarios y BCD, limpiar arreglos de entrada y generar patrones para los dígitos de 7 segmentos. La implementación respeta estrictamente los requerimientos funcionales del enunciado, asegurando que cada modo opere de forma independiente, que las variables se mantengan entre transiciones de modo y que los sensores sean atendidos con precisión. El resultado es un sistema completamente funcional, capaz de administrar un ciclo de entrenamiento real bajo condiciones controladas, ofreciendo mediciones precisas y un despliegue claro y oportuno de la información para el ciclista.

1.2. Tarea_Modo_Espera

La tarea Modo Espera actúa únicamente cuando la variable Funcion indica que el sistema se encuentra en este modo. Cuando esto ocurre, la tarea carga los mensajes “*Velódromo 623*” y “*Modo Espera*” en las líneas del LCD y enciende únicamente el LED correspondiente al modo de espera, asignado en LDEspera. Al mismo tiempo, los dígitos del display de 7 segmentos se apagan, seguido de una llamada a la subrutina BCD_7SEG para actualizar los patrones. Si el sistema no está en este modo, la tarea retorna sin realizar ninguna acción. En este estado no se ejecuta ningún cálculo ni se monitorean sensores, cumpliendo la función de reposo del sistema.

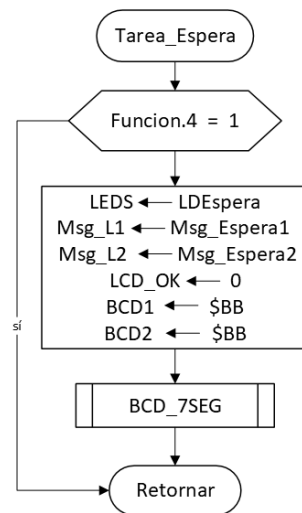


Figura 2: Diagrama de flujo de la tarea modo espera.

1.3. Tarea_Configurar

La tarea Configurar se ejecuta únicamente cuando la variable Funcion indica que el sistema ha entrado en este modo. Su objetivo es permitir al usuario ingresar el número de vueltas mediante el teclado. Al activarse, la tarea enciende el LED correspondiente definido en LDConfig y limpia el arreglo Num_Array y el contador Cont_TCL para garantizar que no existan valores previos. Una vez iniciado el modo, la tarea monitorea la bandera ArrayOK, que indica que el usuario presionó ENTER. Cuando esto ocurre, el valor ingresado es convertido y almacenado en NumVueltas, siempre que se encuentre dentro del rango permitido por MinNumVueltas y MaxNumVueltas. Si el valor no es válido, la tarea simplemente espera nuevos datos sin modificar el sistema. Al completarse una entrada válida, la tarea actualiza Funcion para avanzar al modo Correr. Si el sistema no se encuentra en este modo, la tarea finaliza sin ejecutar cambios.

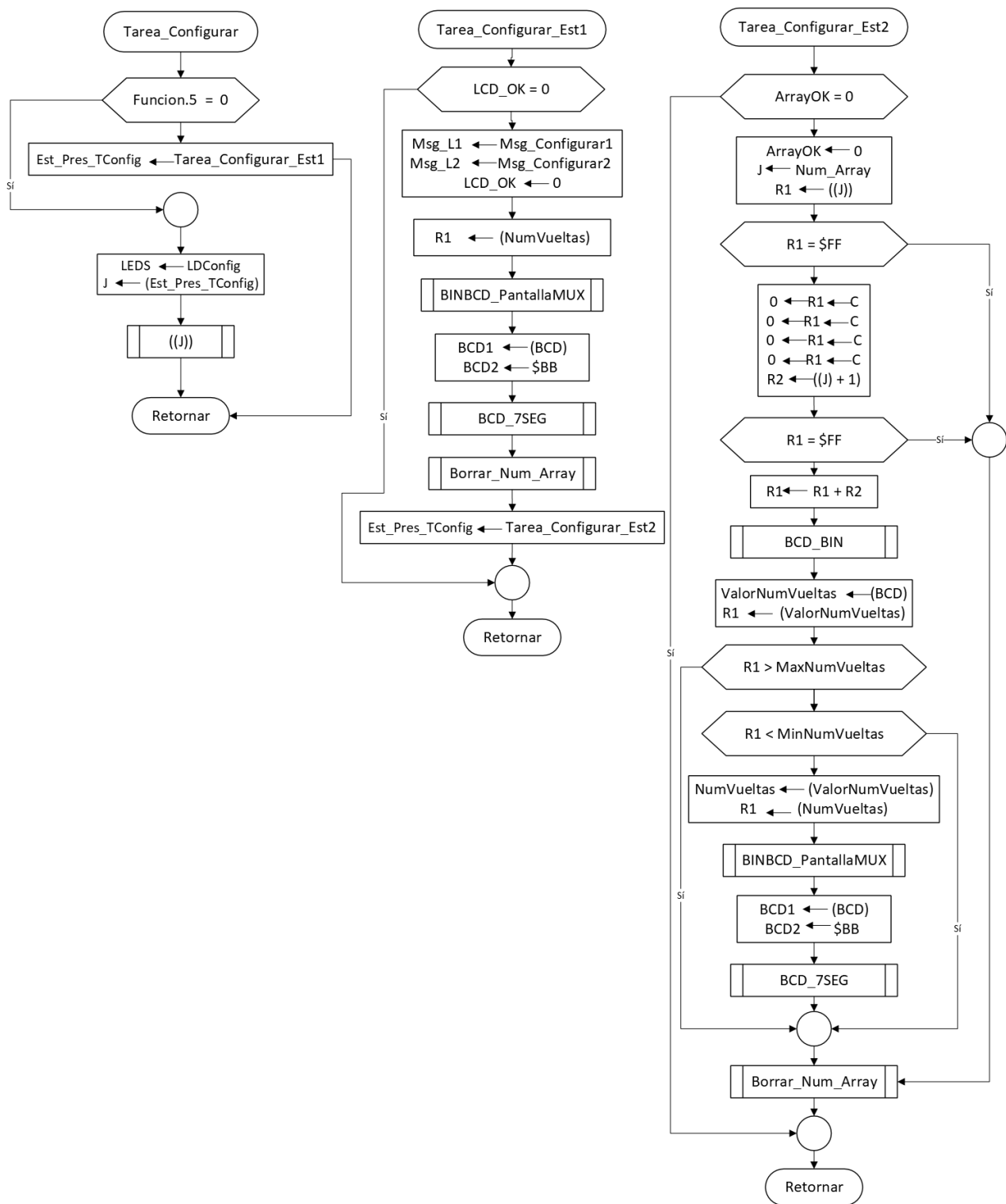


Figura 3: Diagrama de flujo de la tarea modo configuración.

1.4. Tarea_Modo_Correr

La tarea Correr se activa cuando Funcion coincide con el modo Correr. Su propósito es medir el tiempo entre los sensores S1 y S2, calcular la velocidad, controlar el avance de vueltas y manejar los mensajes del LCD. Al ingresar al modo, se enciende el LED definido en LDCorrer y se inicializan los valores DeltaT, Vueltas y temporizadores asociados.

La tarea monitorea las banderas ShortP1 y ShortP2, las cuales representan las activaciones de los sensores simulados. Cuando ShortP1 ocurre, el TimerCal se reinicia, iniciando la medición. Cuando ShortP2 ocurre, se obtiene DeltaT y se llama a la subrutina Calcula para determinar la velocidad y los tiempos de despliegue. Si la velocidad está fuera del rango definido por VMin y VMax, se activa el mensaje de error y se utiliza TimerError para controlar su duración. Si la velocidad es válida, se actualizan AcumVelocidad y Vueltas, y se invoca la conversión a BCD mediante BIN_BCD_MUXP. La tarea continúa repitiendo este ciclo hasta que Vueltas alcanza el valor programado. Cuando esto ocurre, Funcion se actualiza para pasar al modo Resumen. Si el sistema no está en este modo, la tarea retorna sin realizar acciones adicionales.

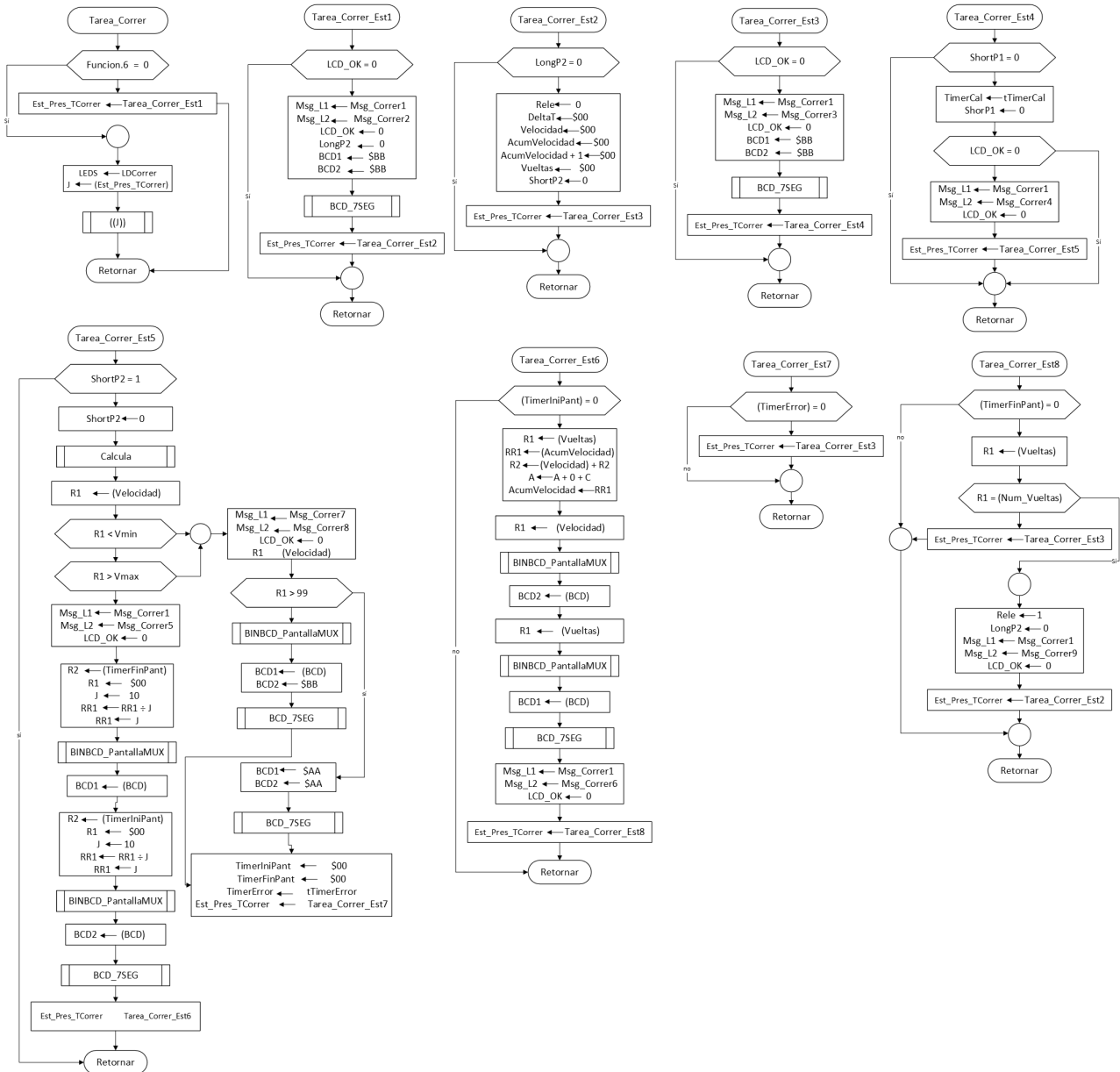


Figura 4: Diagrama de flujo de la tarea modo correr.

1.5. Tarea_Modo_Resumen

La tarea Resumen opera únicamente cuando Funcion indica que el sistema ha completado todas las vueltas. Al activarse, se enciende el LED correspondiente definido en LDResumen y se cargan en el LCD los mensajes que identifican este modo. A continuación, se calcula la velocidad promedio dividiendo AcumVelocidad entre el total de Vueltas realizadas. Estos valores se convierten a BCD mediante BIN_BCD_MUXP y se envían al display para mostrar tanto el número de vueltas como la velocidad promedio. La tarea se mantiene activa mientras el sistema permanezca en este modo y no ejecuta cálculos adicionales. Si Funcion cambia, la tarea finaliza.

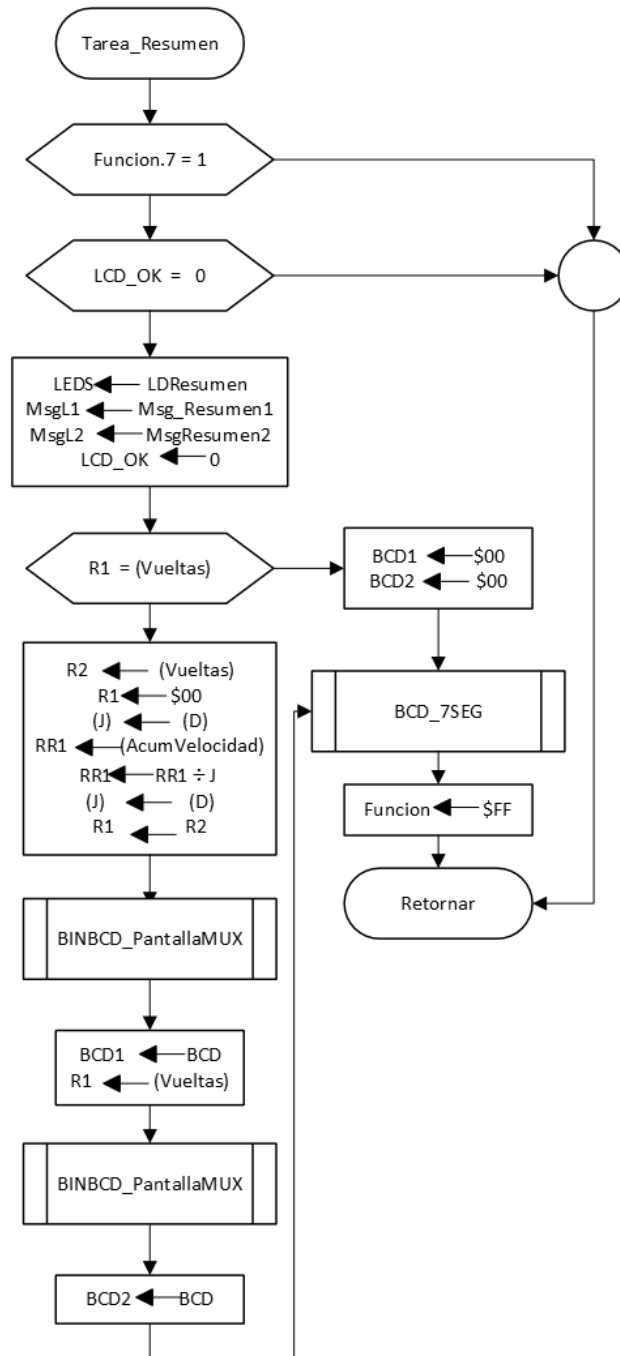


Figura 5: Diagrama de flujo de la tarea modo resumen.

1.6. Tarea_Brillo

La tarea Brillo se encarga de obtener periódicamente el nivel de iluminación mediante el convertidor analógico-digital para ajustar la intensidad del display de 7 segmentos. Esta tarea únicamente actúa cuando el temporizador asignado a la medición, TimerBrillo, ha llegado a cero. Cuando esto ocurre, la tarea inicia un nuevo ciclo de conversión ADC habilitando el módulo correspondiente y borrando la bandera SCF para permitir que una nueva conversión sea registrada.

Una vez que la conversión finaliza, la tarea lee el resultado almacenado y lo asigna a la variable Brillo. Este valor es utilizado posteriormente por la tarea PantallaMUX para modificar la relación de encendido de los dígitos durante el multiplexado, permitiendo que el display se vea más tenue o más brillante según la luz del entorno. Después de actualizar el valor, la tarea recarga TimerBrillo con tTimerBrillo para programar la siguiente medición.

Si el temporizador aún no ha expirado o si el sistema no requiere una nueva lectura, la tarea finaliza sin realizar cambios. De esta forma, el brillo del display se adapta dinámicamente sin interferir con las demás tareas del sistema.

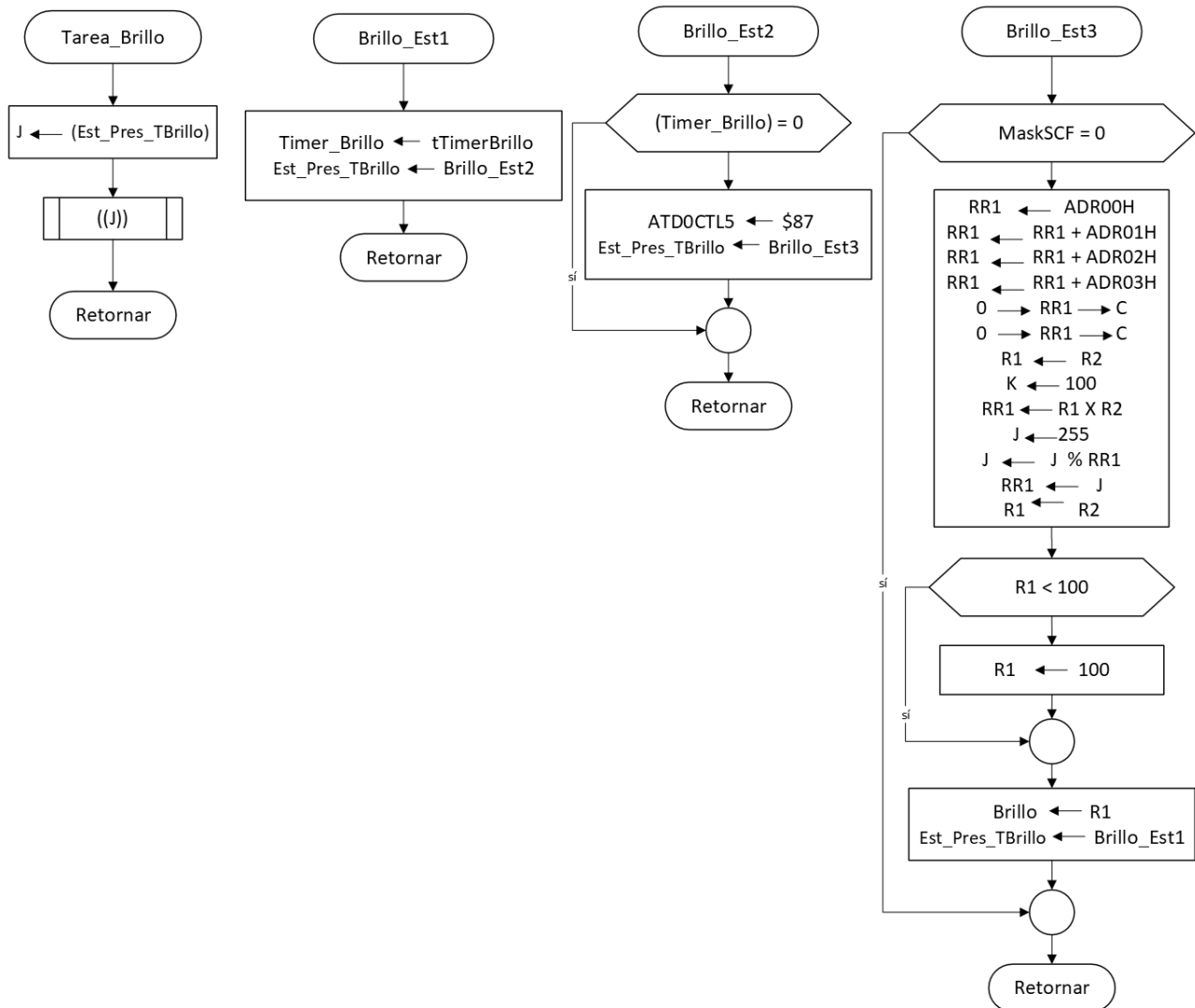


Figura 6: Diagrama de flujo de la tarea brillo.

1.7. Tarea_Teclado

La tarea Teclado se encarga de detectar y procesar las teclas ingresadas por el usuario mediante el teclado matricial. Su funcionamiento se basa en una máquina de estados que coordina el barrido del teclado, la validación del antirrebote y el almacenamiento de los valores ingresados.

En el primer estado, la tarea activa el patrón correspondiente a la fila que se desea leer y verifica si existe alguna tecla presionada. Si no hay actividad, avanza al siguiente patrón para completar el barrido. Si detecta una pulsación, pasa al segundo estado.

El segundo estado aplica el tiempo de supresión de rebote utilizando el temporizador asignado. Cuando el tiempo se cumple, se confirma la tecla y se guarda el valor en Num_Array o en la variable correspondiente. Después de almacenar el valor, la tarea pasa al tercer estado.

En el tercer estado, la tarea identifica si la tecla corresponde a un número, a la tecla de borrar o a la tecla enter. Según el caso, actualiza el contador Cont_TCL, elimina el último dato o indica que la entrada está completa. Finalmente, la tarea regresa al primer estado para continuar monitoreando nuevas teclas.

Este ciclo permite que el sistema capture las entradas del usuario de forma estable y ordenada, garantizando que los modos Configurar y Correr reciban datos válidos y libres de rebotes.

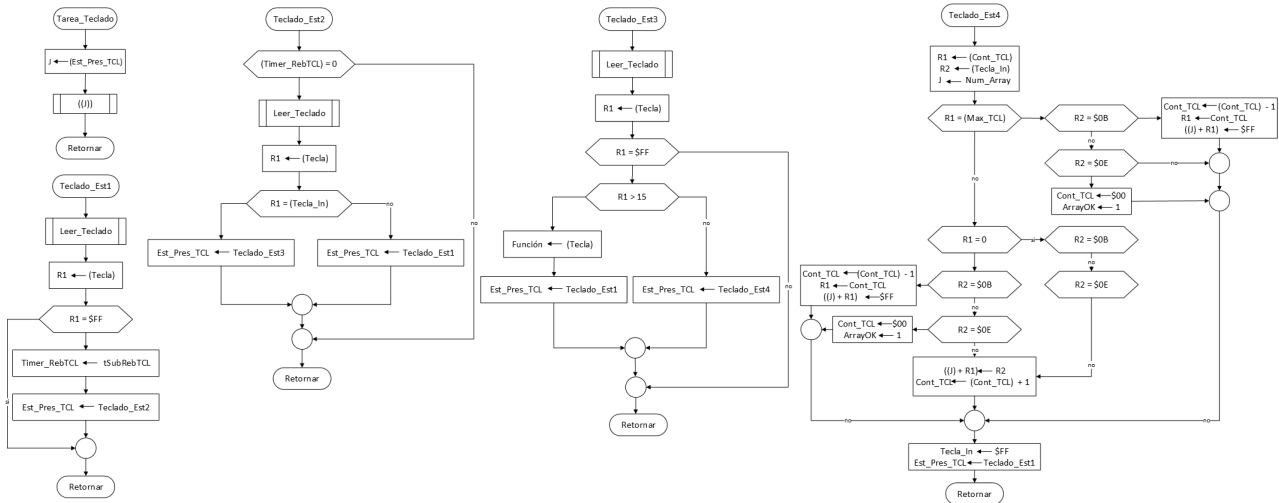


Figura 7: Diagrama de flujo de la tarea teclado.

1.8. Tarea Leer_Teclado

La tarea Leer Teclado se encarga de procesar la tecla que ha sido estabilizada previamente por la Tarea Teclado. Esta tarea actúa únicamente cuando la variable Tecla.IN contiene un valor distinto de cero, indicando que existe una tecla lista para ser interpretada. Una vez detectado este evento, la tarea clasifica la tecla según su tipo: numérica, borrar o ENTER.

Si se trata de una tecla numérica, la tarea la almacena en Num_Array y actualiza el contador Cont_TCL para reflejar la cantidad de dígitos ingresados. Si la tecla corresponde a borrar, se reduce Cont_TCL y se elimina el último valor almacenado. En caso de ser la tecla ENTER, la tarea activa la bandera ArrayOK para indicar que la entrada está completa y lista para ser procesada por otras tareas, como Configurar o Correr.

Después de procesar la tecla, la tarea limpia Tecla.IN para evitar lecturas repetidas y retorna sin realizar acciones adicionales. Si no existe una tecla pendiente, la tarea simplemente termina su ejecución. Este mecanismo asegura que cada tecla ingresada por el usuario sea interpretada de manera ordenada y coherente con el funcionamiento del sistema.

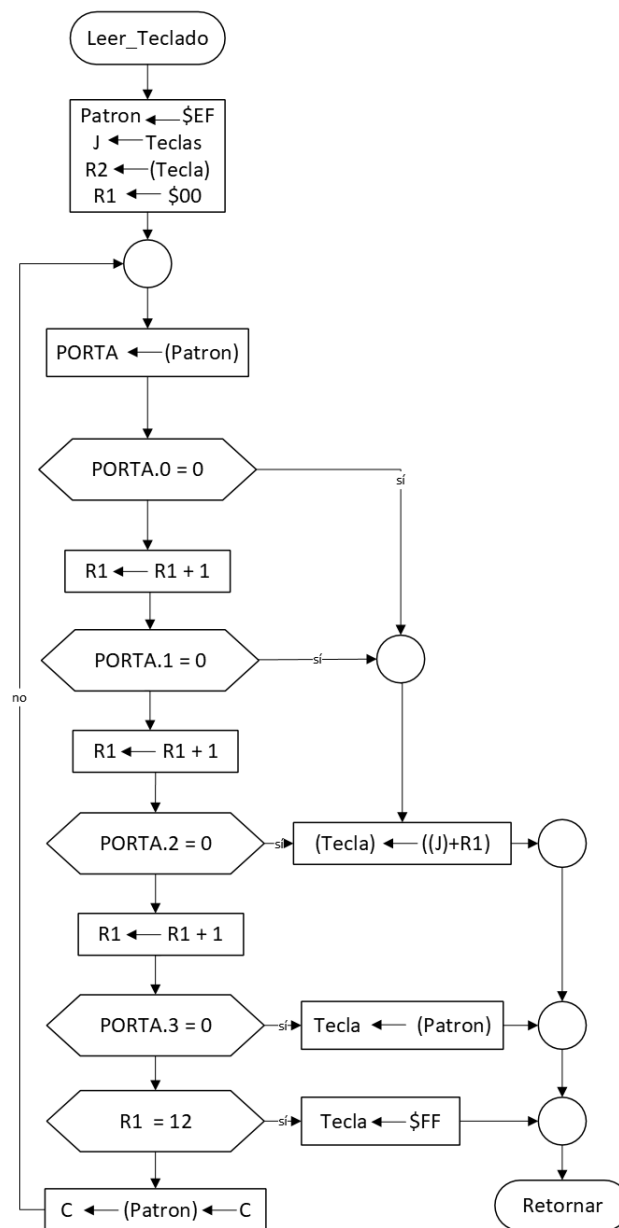


Figura 8: Diagrama de flujo de la tarea leer teclado.

1.9. Tarea_Led_Testigo

La tarea Led Testigo se encarga de controlar el parpadeo del LED indicador del sistema. Esta tarea funciona únicamente cuando el temporizador asignado a este propósito, Timer_LED_Testigo, llega a cero. Cuando esto ocurre, la tarea invierte el estado del LED utilizando la máscara correspondiente, alternando entre encendido y apagado para generar el parpadeo característico.

Después de realizar el cambio de estado del LED, la tarea recarga nuevamente el temporizador con el valor definido en tTimerLDTst, asegurando que el intervalo entre parpadeos se mantenga constante. Si el temporizador aún no ha expirado, la tarea finaliza sin realizar ninguna acción. De esta manera, el LED proporciona una indicación visual del funcionamiento general del sistema sin interferir con el resto de las tareas.

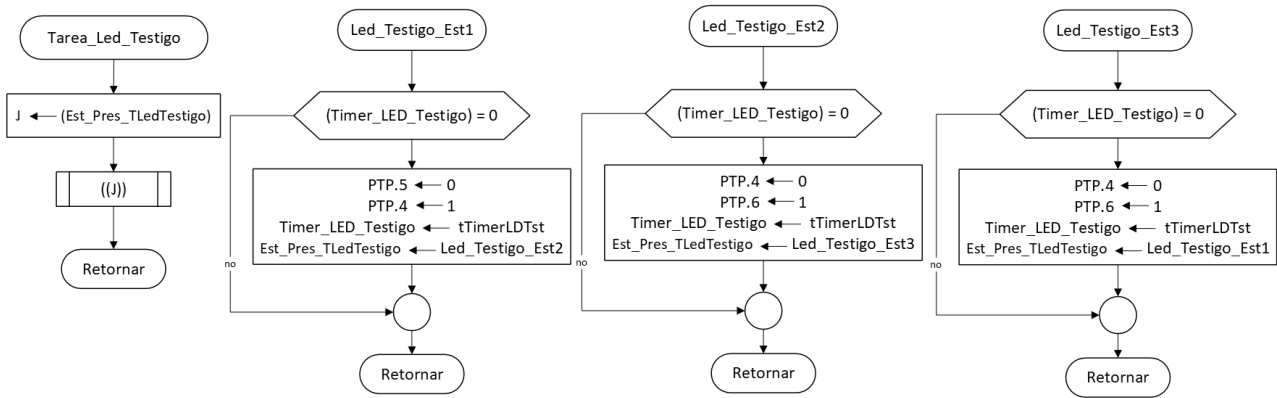


Figura 9: Diagrama de flujo de la tarea led testigo.

1.10. Tarea_Leer_PB1

La tarea Leer PB1 se encarga de identificar el tipo de pulsación realizada sobre el botón PB1, diferenciando entre pulsación corta (ShortP1) y pulsación larga (LongP1). La tarea solo actúa cuando el sistema detecta que PB1 ha sido presionado mediante la máscara correspondiente en el puerto PH.

Una vez detectada la activación, la tarea inicia el temporizador de antirrebote tSupRebPB1. Cuando este tiempo se cumple, se verifica nuevamente el estado del botón para confirmar la pulsación. Si el botón continúa presionado, la tarea inicia el conteo para determinar si se trata de una pulsación corta o larga usando los valores tShortP1 y tLongP1. Si el usuario libera PB1 antes de alcanzar el tiempo mínimo de pulsación larga, se activa la bandera ShortP1. Si el botón se mantiene presionado más allá del tiempo asignado para una pulsación larga, se activa LongP1.

Una vez registrada la acción, la tarea limpia los temporizadores asociados y retorna al estado inicial. Si no se detecta presión o si el rebote aún no ha finalizado, la tarea continúa en espera sin modificar el sistema. Esto permite diferenciar correctamente las acciones del usuario según la duración de la pulsación.

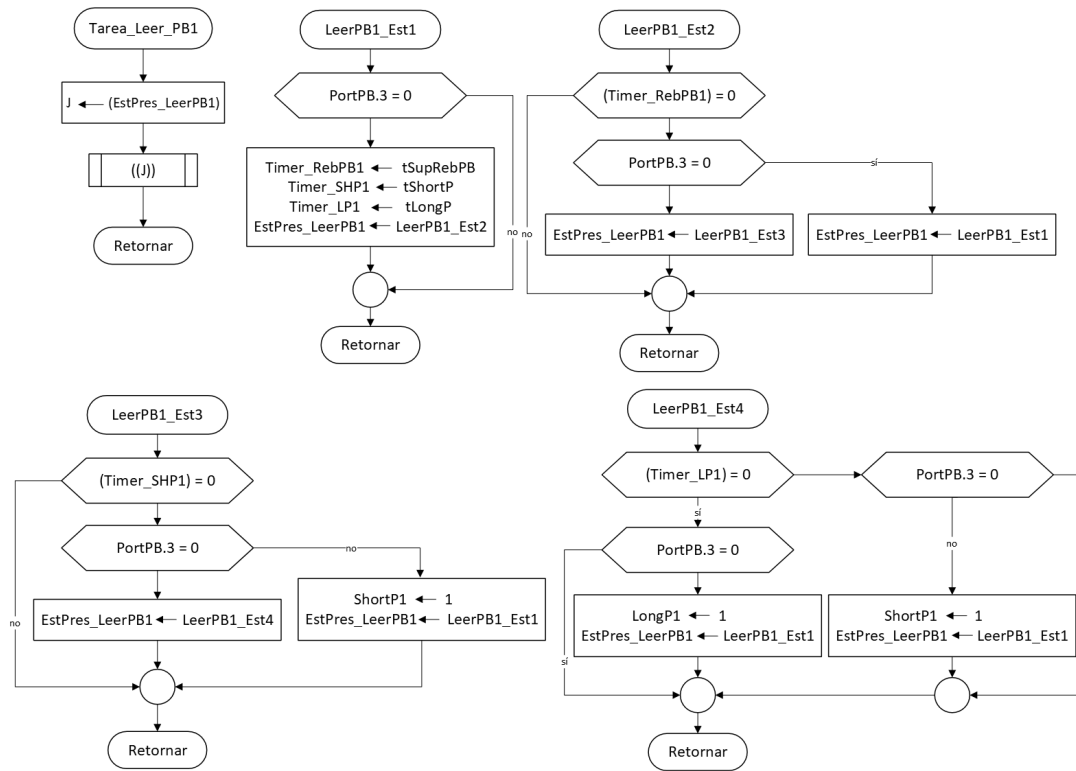


Figura 10: Diagrama de flujo de la tarea leer PB1.

1.11. Tarea Leer_PB2

La tarea Leer PB2 es totalmente análoga a la tarea Leer PB1, pero enfocada en el sensor PB2.

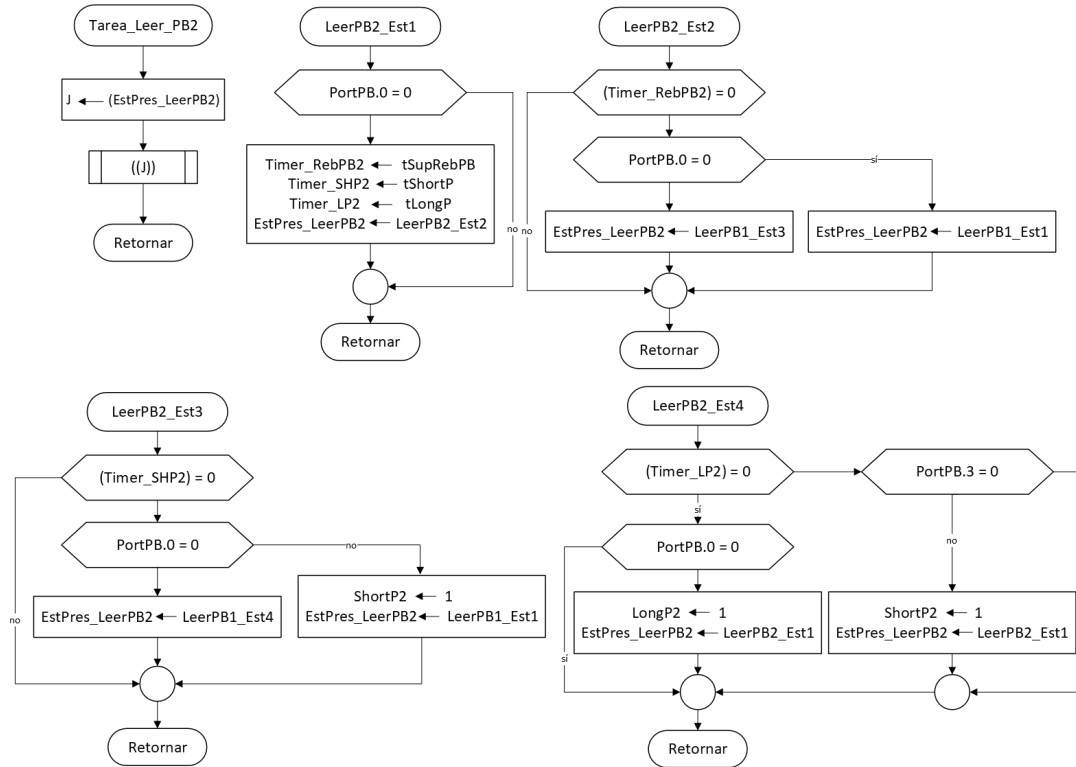


Figura 11: Diagrama de flujo de la tarea leer PB2.

1.12. Tarea_PantallaMUX

La tarea PantallaMUX se encarga del control del display de 7 segmentos mediante multiplexado. Su función principal es activar de forma secuencial cada uno de los dígitos utilizando Cont_Dig como índice, y enviar el patrón de segmentos correspondiente almacenado en Dsp1, Dsp2, Dsp3 o Dsp4. De esta forma, aunque solo un dígito está encendido a la vez, la velocidad de conmutación genera la percepción de que los cuatro están activos simultáneamente.

Cada vez que TimerDigito llega a cero, la tarea avanza al siguiente dígito del display. Dependiendo del valor de Cont_Dig, activa la máscara apropiada (DIG1, DIG2, DIG3 o DIG4) y coloca en el puerto correspondiente el patrón de segmentos definido previamente por la subrutina BCD_7SEG o por las tareas que actualizan los valores a desplegar. Si se requiere apagar el display, por ejemplo en el modo Espera o ciertos estados de Correr, la tarea asigna patrones nulos antes de continuar el ciclo.

Una vez actualizado el dígito, la tarea recarga TimerDigito utilizando tTimerDigito y avanza el contador para refrescar el display. Si el temporizador aún no ha expirado, la tarea retorna sin modificar el estado del display. De esta forma, el sistema mantiene un refresco continuo, estable y ajustado al brillo definido por la tarea Brillo, asegurando una visualización clara en todos los modos de operación.

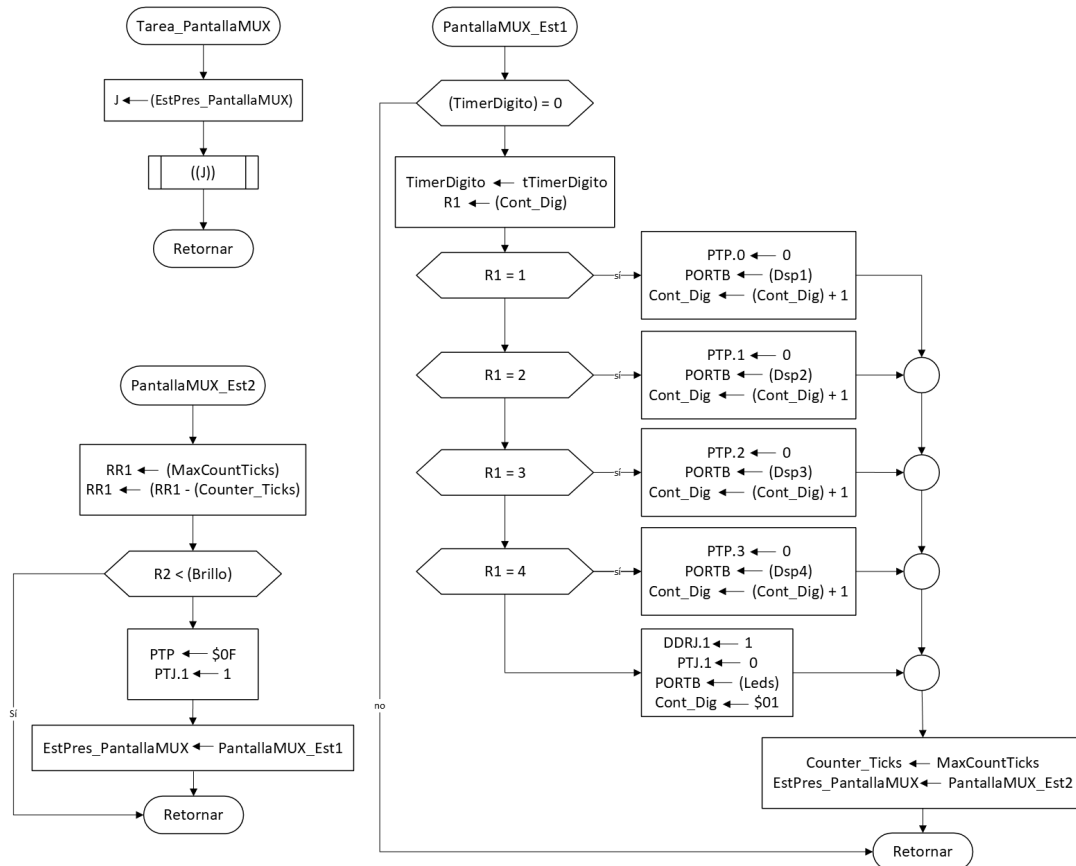


Figura 12: Diagrama de flujo de la tarea pantalla MUX.

1.13. Tarea_LCD

La tarea LCD es la encargada de gestionar el envío de mensajes al módulo de visualización LCD utilizando el modo de transferencia de 4 bits. Esta tarea actúa únicamente cuando el sistema requiere actualizar el contenido del display, ya sea porque una tarea cambió los mensajes activos o porque se solicitó un nuevo despliegue al cambiar de modo. La tarea utiliza las variables Msg_L1, Msg_L2 y Punt_LCD para determinar qué mensaje debe enviarse y en qué posición se encuentra el proceso de escritura.

Cuando la tarea es invocada, primero verifica si el LCD está listo para recibir un nuevo comando consultando la bandera LCD_OK. Si la bandera indica que el módulo aún está ocupado, la tarea retorna sin realizar ninguna acción. Si el módulo está disponible, la tarea determina si debe enviar un comando o un dato, asignando el valor correspondiente a RS y colocando en CharLCD la porción del mensaje que corresponde enviar en ese ciclo.

La tarea también gestiona el avance del puntero Punt_LCD, el cual indica el siguiente carácter del mensaje a desplegar. Una vez que se completa una línea, se activa la señal Second_Line para indicar que el controlador debe cambiar a la segunda línea antes de continuar. Si ambas líneas ya fueron enviadas, la tarea finaliza el proceso y permite que otras tareas modifiquen nuevamente el contenido del LCD.

Si no existe ningún mensaje pendiente por enviar o si el sistema no ha solicitado un nuevo despliegue, la tarea retorna sin modificar el estado del módulo. De esta forma, la tarea LCD asegura una comunicación ordenada y estable con el display, evitando conflictos entre tareas y garantizando que el mensaje visualizado siempre corresponda al modo de operación actual.

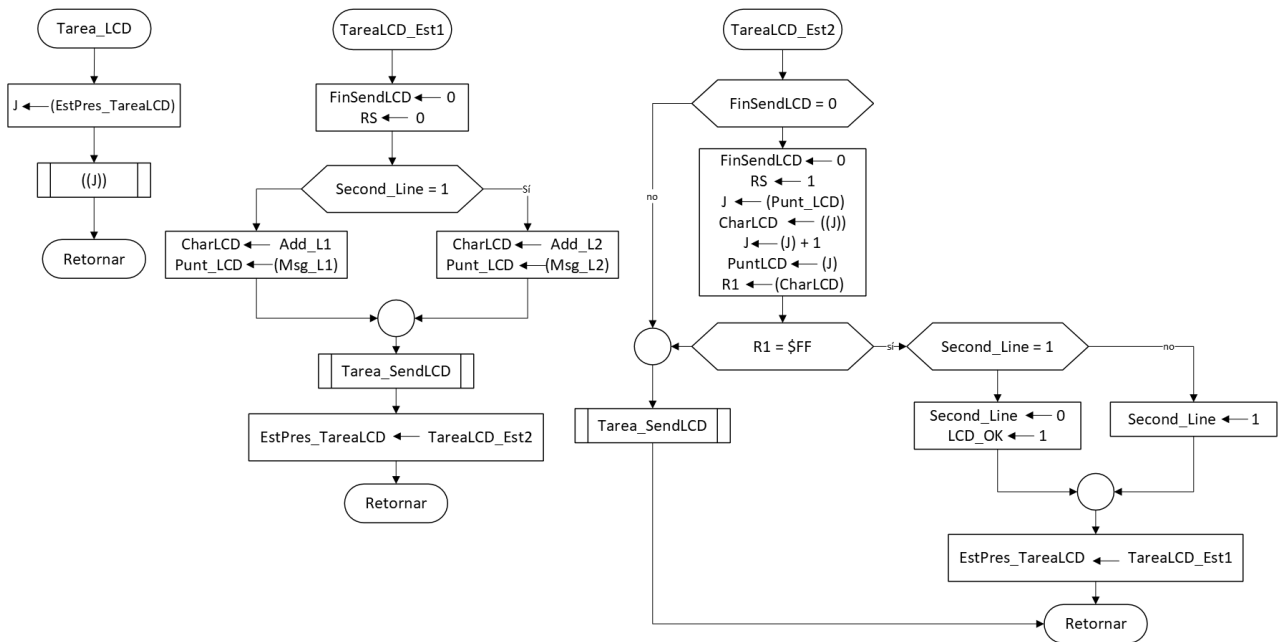


Figura 13: Diagrama de flujo de la tarea LCD.

1.14. Tarea_Send_LCD

La tarea SendLCD es la encargada de realizar físicamente el envío de cada nibble de información hacia el módulo LCD utilizando el modo de transferencia de 4 bits. A diferencia de la Tarea LCD, que decide qué carácter o comando debe enviarse, SendLCD ejecuta directamente la secuencia eléctrica necesaria para transmitirlo al hardware. Esta tarea actúa únicamente cuando se detecta que hay un dato pendiente en CharLCD y que el módulo está habilitado para continuar la escritura.

Cuando SendLCD es invocada, primero verifica que la bandera LCD_OK esté en cero, lo que indica que el LCD está listo para recibir un nuevo nibble. Si LCD_OK aún no ha sido limpiada, la tarea retorna sin ejecutar la transferencia. Una vez validado el estado, la tarea divide CharLCD en sus dos mitades: el nibble alto y el nibble bajo. Cada nibble se coloca en el puerto correspondiente, y a continuación se genera el pulso de habilitación utilizando los temporizadores tTimer260uS y tTimer40uS para asegurar los tiempos mínimos requeridos por el controlador del LCD.

El valor de RS se utiliza para definir si el dato enviado corresponde a un comando o a un carácter de texto. Después de completar el envío del nibble, la tarea activa la bandera FinSendLCD para indicar a la Tarea LCD que la transferencia finalizó correctamente y que puede avanzar al siguiente carácter del mensaje. Si todavía quedan datos por enviar, la tarea repetirá el procedimiento hasta completar el carácter completo.

Si no existe información pendiente o si el LCD aún está ocupado, SendLCD termina sin realizar ninguna modificación. Este mecanismo garantiza una comunicación estable y sincronizada con el dispositivo, evitando errores en el despliegue y asegurando que cada carácter llegue correctamente al módulo LCD.

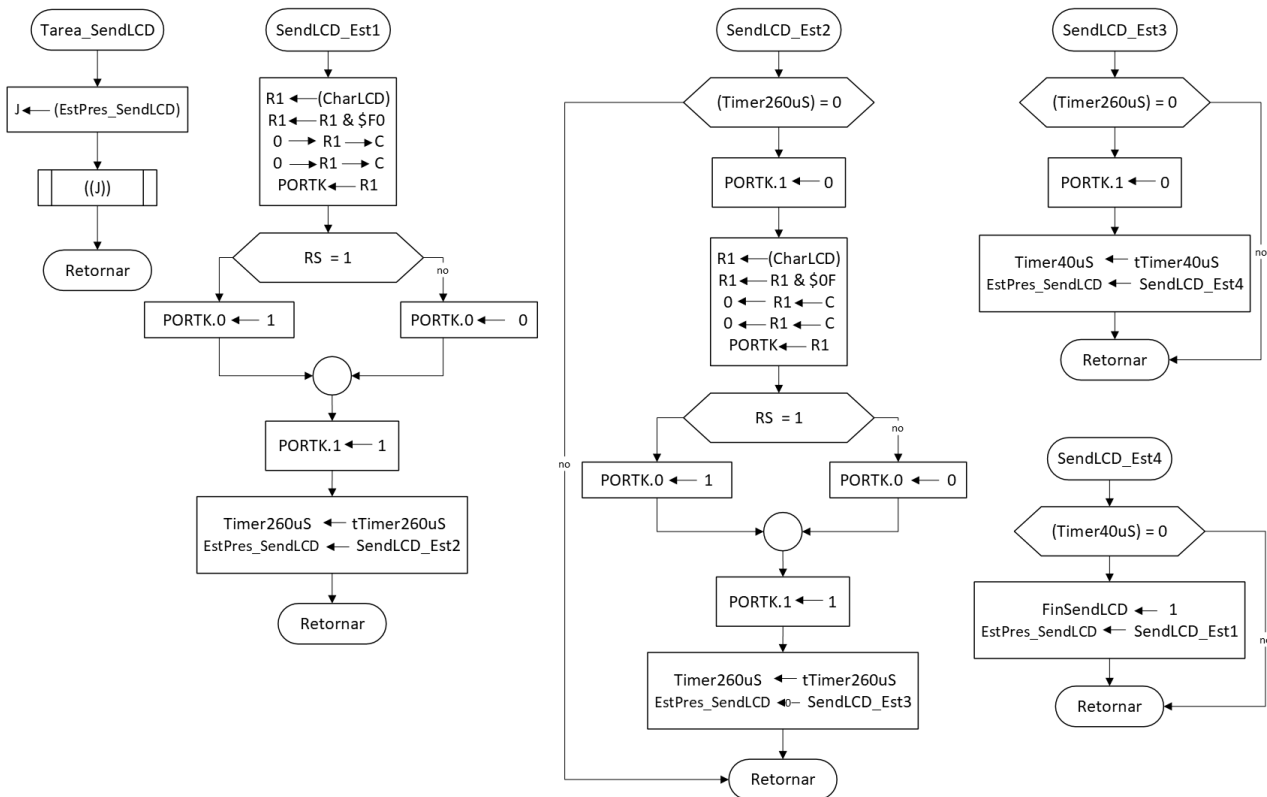


Figura 14: Diagrama de flujo de la tarea send LCD.

1.15. Tarea_BCD_BIN

Cuando la subrutina es llamada, toma como entrada el valor binario que desea convertirse y ejecuta un proceso de división sucesiva para separar las decenas y unidades. El resultado parcial de la conversión se almacena en la variable BCD, mientras que los dígitos resultantes se guardan en BCD1 y BCD2. BCD1 contiene el dígito de las decenas y BCD2 el de las unidades. Una vez calculados, estos valores son utilizados posteriormente por la Tarea PantallaMUX para asignar los patrones segmentados correspondientes en Dsp1 y Dsp2, o en los dígitos del display que corresponda según el modo actual.

La subrutina finaliza cuando ambos dígitos han sido generados y coloca los resultados en las variables asociadas, sin modificar el valor original de entrada. De esta manera, BCD_BIN asegura que cualquier valor numérico producido por el sistema pueda visualizarse correctamente en el display de 7 segmentos.

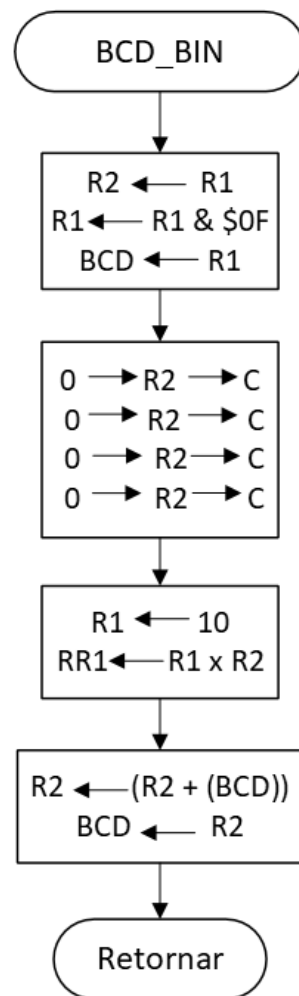


Figura 15: Diagrama de flujo de la tarea BCD BIN.

1.16. Subrutina Calcula

La subrutina Calcula se encarga de determinar la velocidad del ciclista y los tiempos asociados al despliegue en pantalla utilizando el intervalo medido entre los sensores S1 y S2. Esta rutina recibe como entrada el valor DeltaT, que corresponde al tiempo transcurrido entre ambos sensores, previamente capturado por la tarea Correr mediante TimerCal.

El primer paso de la subrutina consiste en verificar que DeltaT se encuentre dentro de un rango válido de operación. Si DeltaT es menor que DeltaS o mayor que DeltaP, la subrutina clasifica la medición como fuera de rango y activa la condición correspondiente para que la tarea Correr despliegue el mensaje de error. En estos casos, no se realiza ningún cálculo adicional y la rutina retorna de inmediato.

Cuando DeltaT es válido, Calcula procede a convertir el tiempo en una velocidad proporcional basándose en la longitud del segmento entre los sensores y las escalas establecidas por el sistema. El resultado se almacena en la variable Velocidad, y además se incrementa AcumVelocidad para permitir el cálculo de promedios al finalizar las vueltas.

La subrutina también determina los tiempos de visualización que utiliza la tarea Correr: TimerIniPant y TimerFinPant. Estos temporizadores permiten mostrar primero los datos de inicio y luego los datos finales después de cada paso por el sensor. Ambos valores dependen directamente de la velocidad obtenida, de modo que las pantallas se adapten a la dinámica real del ciclista.

Finalmente, la subrutina retorna sin modificar los valores originales de entrada, dejando que las tareas correspondientes se encarguen del despliegue en pantalla y la actualización de vueltas. De esta forma, Calcula constituye el núcleo del procesamiento numérico del sistema, transformando la lectura de sensores en información útil para el usuario.

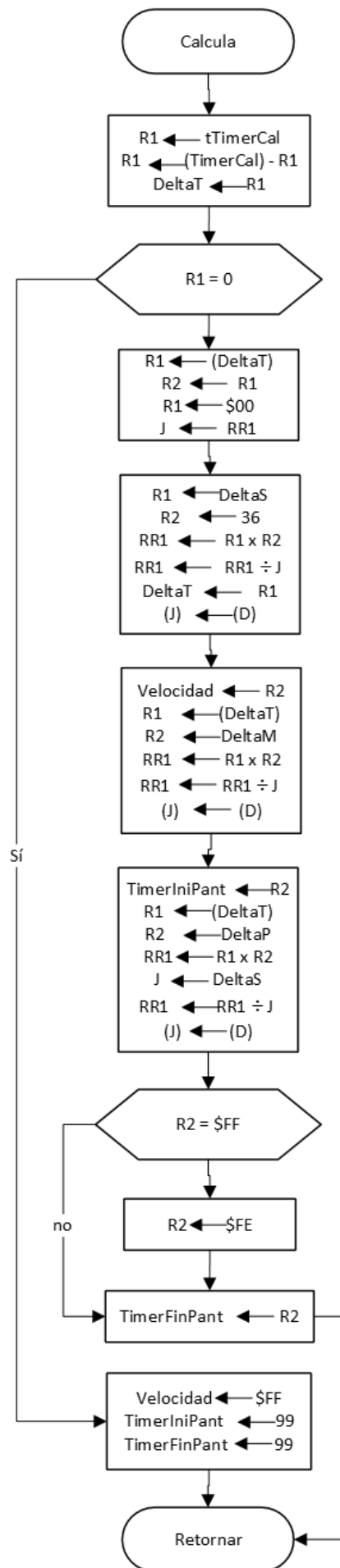


Figura 16: Diagrama de flujo de la subrutina calcula.

1.17. Subrutina BIN_BCD_MUXP

La subrutina BIN_BCD_MUXP convierte un valor binario en sus dígitos BCD y los coloca en los espacios de despliegue utilizados por la Tarea PantallaMUX. Primero separa el valor en decenas y unidades, almacenándolas en BCD1 y BCD2. Luego asigna estos dígitos a Dsp1, Dsp2, Dsp3 o Dsp4 según el dato que se desea mostrar, como velocidad o número de vueltas. Finalmente, convierte cada dígito a su patrón correspondiente de 7 segmentos para que PantallaMUX pueda actualizar el display sin realizar cálculos adicionales.

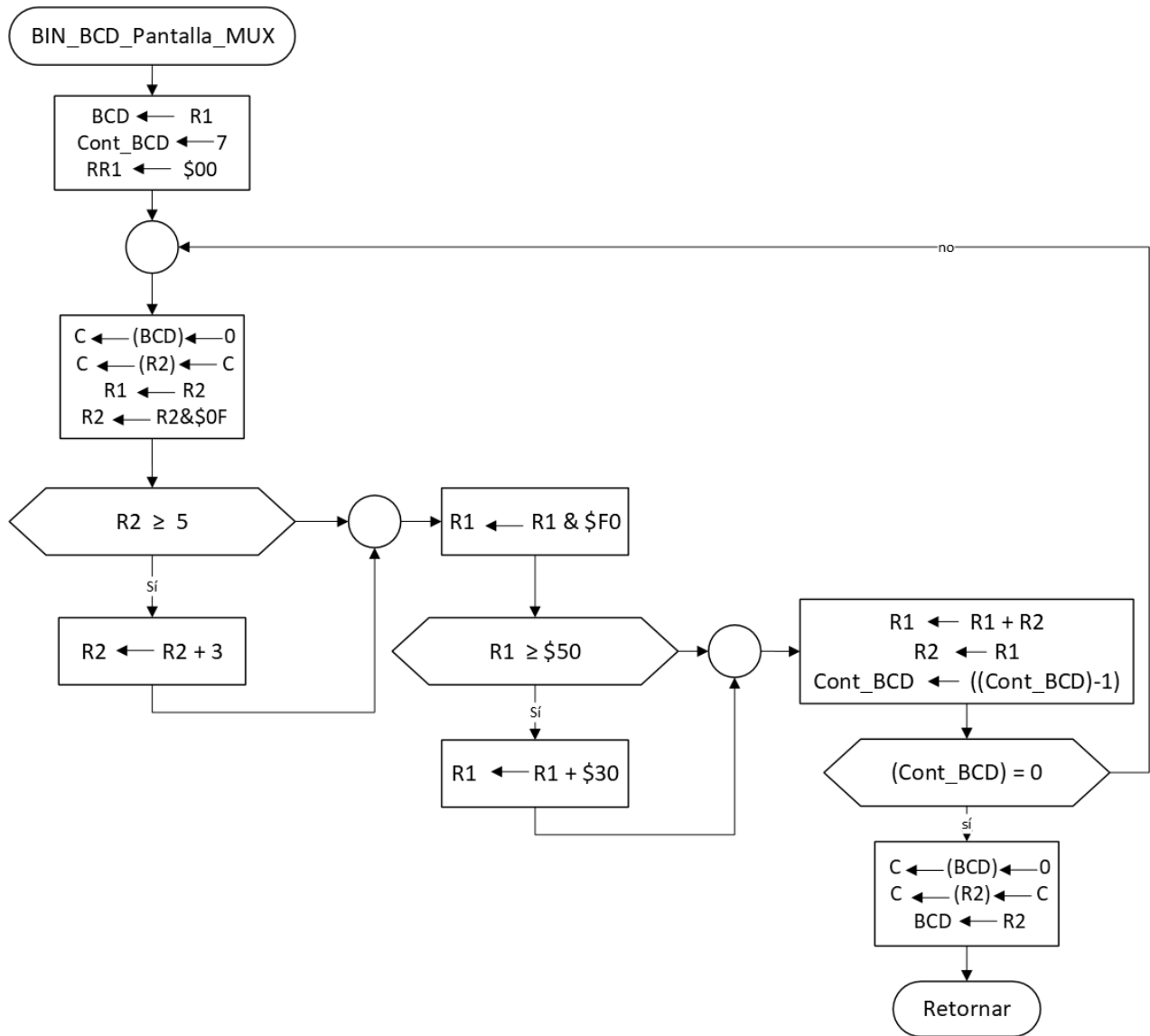


Figura 17: Diagrama de flujo de la subrutina BIN BCD pantalla MUX.

1.18. Subrutina BCD_7SEG

La subrutina BCD_7SEG se encarga de convertir un dígito en formato BCD a su patrón equivalente de 7 segmentos utilizando la tabla Segment. El valor BCD recibido se usa como índice para seleccionar el patrón correspondiente, el cual luego se almacena en la variable del dígito que será desplegado (Dsp1, Dsp2, Dsp3 o Dsp4). Esta subrutina no realiza cálculos adicionales; simplemente traduce cada dígito a su representación visual para que la Tarea PantallaMUX pueda enviarlo al display.

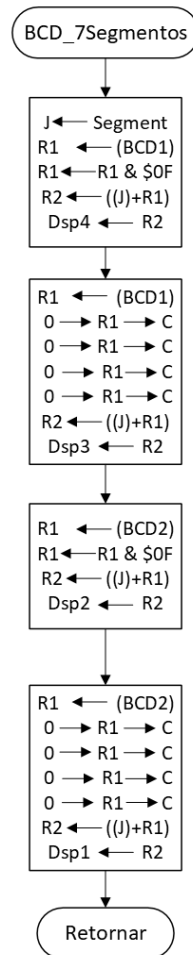


Figura 18: Diagrama de flujo de la subrutina BCD 7SEG.

1.19. Subrutina Borrar_NumArray

La subrutina BorrarNumArray se encarga de limpiar el arreglo Num_Array antes de ingresar un nuevo valor desde el teclado. Para ello, recorre todas las posiciones del arreglo y las coloca en cero, asegurando que no queden dígitos residuales de ciclos anteriores. También reinicia el contador Cont_TCL para indicar que no hay teclas almacenadas.

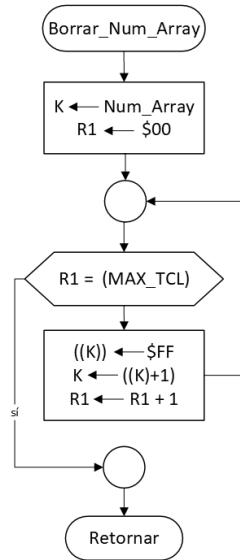


Figura 19: Diagrama de flujo de la subrutina borrar num array.

1.20. Máquina de Tiempos

La máquina de tiempos se encarga de generar las bases de temporización utilizadas por todas las tareas del sistema. En cada interrupción del temporizador, decrementa los contadores activos como Timer1mS, Timer10mS, Timer100mS y los diferentes temporizadores asignados a pulsadores, brillo, multiplexado y control del LCD. Cuando alguno de estos contadores llega a cero, se habilitan las acciones correspondientes en las tareas asociadas. De esta manera, la máquina de tiempos sincroniza todo el sistema sin depender de retardos por software y permite que cada tarea se ejecute en el momento exacto que le corresponde.

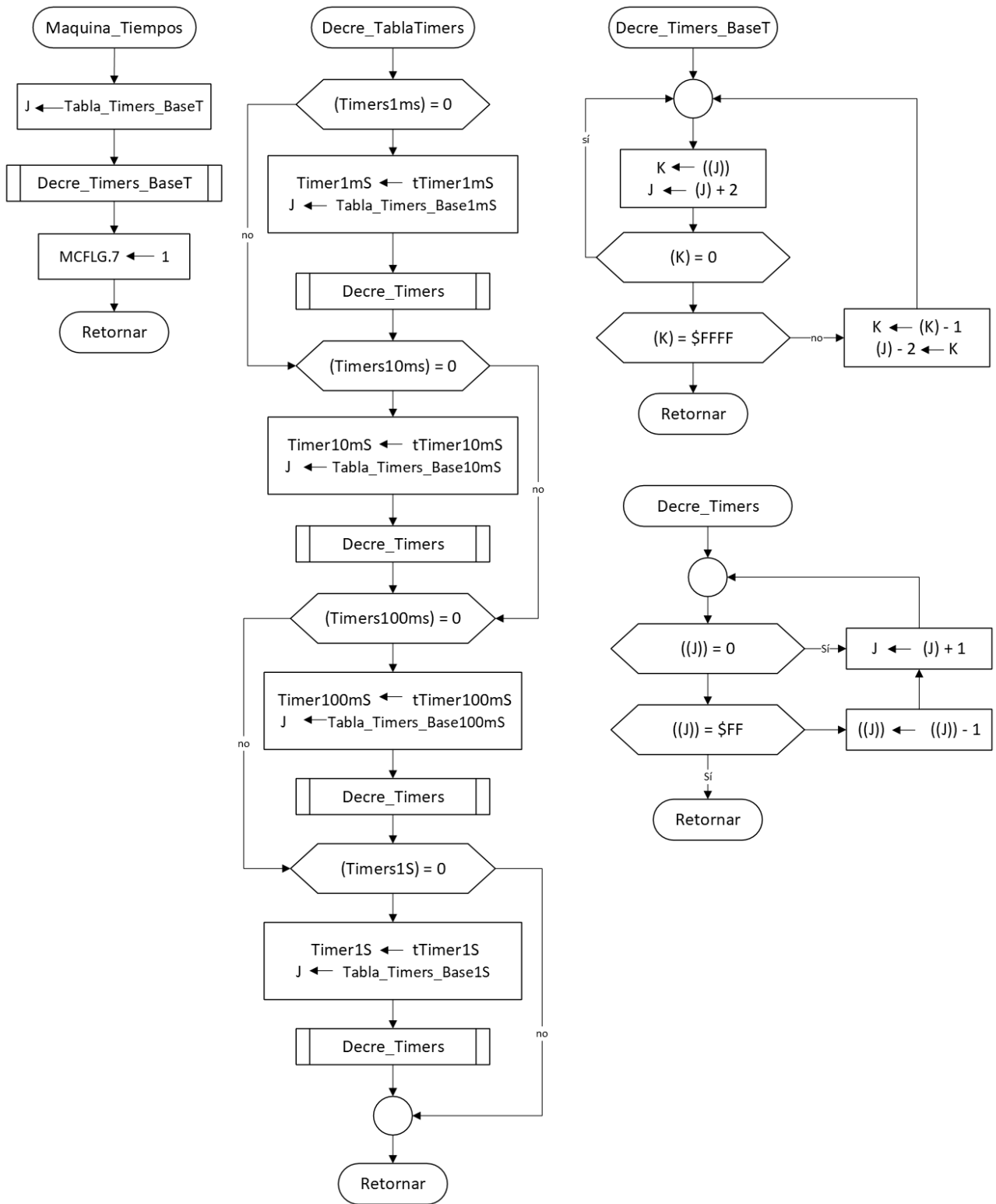


Figura 20: Diagrama de flujo de la máquina de tiempos.

2. Estructuras de datos

Variables de la Tarea Teclado

Tabla 1: Variables asociadas al teclado matricial	
Variable	Descripción
MAX_TCL	Máxima cantidad de teclas permitidas en la entrada
Tecla	Última tecla presionada detectada por el sistema
Tecla.IN	Tecla procesada tras aplicar antirrebote
Cont_TCL	Contador de teclas válidas ingresadas
Patron	Patrón de barrido utilizado para leer el teclado matricial
Funcion	Indica el modo o función activa del sistema
Est_Pres_TCL	Estado presente de la máquina de estados de la Tarea Teclado
Num_Array	Arreglo donde se almacenan los dígitos ingresados por el usuario

Tarea PantallaMUX

Tabla 2: Variables asociadas a la Tarea PantallaMUX	
Variable	Descripción
Est_Pres_PantallaMUX	Estado actual de la tarea PantallaMUX
Dsp1	Patrón de segmentos para el dígito 1
Dsp2	Patrón de segmentos para el dígito 2
Dsp3	Patrón de segmentos para el dígito 3
Dsp4	Patrón de segmentos para el dígito 4
LEDS	Control de LEDs de modo en el display
Cont_Dig	Índice del dígito actualmente habilitado
Brillo	Valor del brillo aplicado al multiplexado

Variables para subrutinas de Conversión

Tabla 3: Variables utilizadas en las subrutinas de conversión BCD	
Variable	Descripción
BCD	Resultado parcial de la conversión binario-BCD
Cont_BCD	Contador usado en el algoritmo de conversión
BCD1	Primer dígito convertido en formato BCD (decenas)
BCD2	Segundo dígito convertido en formato BCD (unidades)

Tarea LCD

Tabla 4: Variables asociadas a la Tarea LCD

Variable	Descripción
IniDsp	Inicialización del LCD y parámetros de arranque
Punt_LCD	Puntero al carácter actual del mensaje a enviar
CharLCD	Byte a enviar al controlador LCD (modo 4 bits)
Msg_L1	Dirección del mensaje de la línea 1
Msg_L2	Dirección del mensaje de la línea 2
EstPres_SendLCD	Estado presente de la Tarea SendLCD
EstPres_TareaLCD	Estado presente de la Tarea LCD

Tarea Leer PB1 y Tarea Leer PB2

Tabla 5: Variables asociadas a la lectura de botones PB1 y PB2

Variable	Descripción
EstPres_LeerPB1	Estado presente de la máquina PB1
EstPres_LeerPB2	Estado presente de la máquina PB2

Tarea Configurar

Tabla 6: Variables asociadas a la Tarea Configurar

Variable	Descripción
Est_Pres_TConfig	Estado presente de la Tarea Configurar
ValorNumVueltas	Valor ingresado desde el teclado antes de validarse
NumVueltas	Número de vueltas programadas por el usuario

Tarea Correr

Tabla 7: Variables asociadas a la Tarea Correr

Variable	Descripción
Est_Pres_TCorrer	Estado presente de la Tarea Correr
DeltaT	Tiempo entre sensores S1 y S2
Velocidad	Velocidad instantánea calculada
AcumVelocidad	Acumulado de velocidades para promedio
Vueltas	Vueltas completadas

Tarea Brillo

Tabla 8: Variables asociadas a la Tarea Brillo

Variable	Descripción
Est_Pres_TBrillo	Estado presente de la Tarea Brillo

Tarea Led Testigo

Tabla 9: Variables asociadas a la Tarea Led Testigo

Variable	Descripción
Es_Pres_TLedTestigo	Estado presente de la Tarea Led Testigo

Banderas

Tabla 10: Banderas utilizadas por el sistema

Bandera	Descripción
ShortP1	Pulsación corta detectada en PB1
LongP1	Pulsación larga detectada en PB1
ShortP2	Pulsación corta detectada en PB2
LongP2	Pulsación larga detectada en PB2
ArrayOK	Indica que la entrada desde el teclado está completa
RS	Selección de registro para el LCD (comando/dato)
LCD_Ok	El LCD terminó la operación de escritura
FinSendLCD	La tarea SendLCD finalizó el envío del carácter actual
Second_Line	Indica que el LCD debe escribir en la segunda línea

Generales

Tabla 11: Variables generales del sistema

Variable	Descripción
LED_Testigo	LED indicador del estado de funcionamiento
Carga_TC4	Valor de carga para temporización asociada al testigo

Tabla Segment

Tabla 12: Patrones de 7 segmentos

Entrada	Descripción
3F	Dígito '0' en 7 segmentos
06	Dígito '1' en 7 segmentos
5B	Dígito '2' en 7 segmentos
4F	Dígito '3' en 7 segmentos
66	Dígito '4' en 7 segmentos
6D	Dígito '5' en 7 segmentos
7D	Dígito '6' en 7 segmentos
07	Dígito '7' en 7 segmentos
7F	Dígito '8' en 7 segmentos
6F	Dígito '9' en 7 segmentos
40	Carácter '-'
00	Espacio en blanco ' '

Tabla Teclas

Tabla 13: Valores asociados a cada tecla del teclado matricial

Valor	Descripción
01	Tecla 1
02	Tecla 2
03	Tecla 3
04	Tecla 4
05	Tecla 5
06	Tecla 6
07	Tecla 7
08	Tecla 8
09	Tecla 9
0B	Tecla BORRAR
00	Sin tecla / vacío
0E	Tecla ENTER

Mensajes del LCD

Tabla 14: Mensajes predefinidos para despliegue en LCD

Mensaje	Contenido
Msg_Espera1	"*VELODROMO 623*"
Msg_Espera2	"**MODO ESPERA**"
Msg_Configurar1	"MODO CONFIGURAR"
Msg_Configurar2	"* NUM VUELTAS *"
Msg_Correr1	"**MODO CORRER**"
Msg_Correr2	". ^{ES} PERANDO INICIO"
Msg_Correr3	". ^{ES} PERANDO S1... "
Msg_Correr4	". ^{ES} PERANDO S2... "
Msg_Correr5	"TIniP TFinP"
Msg_Correr6	"VELOC. VUELTAS"
Msg_Correr7	"** VELOCIDAD **"
Msg_Correr8	"*FUERA DE RANGO*"
Msg_Correr9	"**FIN DE CICLO**"
Msg_Resumen1	"MODO RESUMEN "
Msg_Resumen2	"VUELTAS VELOC"

Tabla de Timers

Tabla 15: Timers utilizados en el sistema

Timer	Descripción
Timer1mS	Contador base de 1 ms.
Timer10mS	Contador con base 10 ms, derivado del Timer1mS.
Timer100mS	Base de 100 ms utilizada en cálculos de velocidad, actualización de estados y temporizaciones perceptibles para el usuario.
Timer1S	Contador equivalente a 1 segundo para tareas de larga duración y eventos extendidos.
CounterTicks	Contador de ticks para administrar el multiplexado de los dígitos del display de 7 segmentos. Controla los tiempos de encendido y estabilidad visual.
Timer260uS	Retardo de 260 μ s empleado para generar el pulso de habilitación del LCD en modo de 4 bits. Garantiza la correcta lectura del comando.
Timer40uS	Retardo mínimo de 40 μ s requerido entre comandos del LCD para asegurar la correcta ejecución del controlador.
Timer_RebPB1	Tiempo de supresión de rebote del pulsador PB1.
Timer_RebPB2	Tiempo de supresión de rebote del pulsador PB2.
Timer_RebTCL	Tiempo de supresión de rebotes del teclado matricial. Permite estabilizar la tecla antes de procesarla.
TimerDigito	Tiempo durante el cual cada dígito de 7 segmentos permanece encendido en el multiplexado.
Timer2mS	Retardo de 2 ms utilizado principalmente en el comando <i>Clear LCD</i> , permitiendo que el controlador complete la instrucción antes de continuar.
Timer_SHP1	Tiempo mínimo para reconocer una pulsación corta en el PB1.
Timer_SHP2	Tiempo mínimo para reconocer una pulsación corta en el PB2.
TimerCal	Timer utilizado durante el cálculo de velocidad. Su base permite medir el intervalo entre S1 y S2.
TimerError	Tiempo durante el cual se mantiene en pantalla el mensaje de “Fuera de Rango”.
TimerIniPant	Tiempo inicial antes de mostrar información en la pantalla durante ciertos estados.
TimerFinPant	Tiempo que debe permanecer visible la información antes de volver al modo previo.
TimerBrillo	Timer utilizado para definir cuándo debe iniciarse un nuevo ciclo de conversión ADC para determinar el nivel de brillo.
Timer_LED_Testigo	Temporizador que regula el parpadeo del LED testigo.
Timer_LP1	Tiempo asignado para detectar una pulsación larga en PB1.
Timer_LP2	Tiempo asignado para detectar una pulsación larga en PB2.

Máscaras y constantes del sistema

Tabla 16: Máscaras y constantes utilizadas en el sistema

Constante / Máscara	Descripción
tSupRebTCL	Tiempo de supresión de rebote del teclado (20 ms).
tTimerDigito	Tiempo por dígito en el multiplexado (2 ms).
MaxCountTicks	Cantidad máxima de ciclos para refrescar pantalla (100).
DIG1	Selección del dígito 1 del display.
DIG2	Selección del dígito 2 del display.
DIG3	Selección del dígito 3 del display.
DIG4	Selección del dígito 4 del display.
tTimer2mS	Retardo de 2 ms usado por Clear LCD.
tTimer260uS	Retardo de 260 μ s necesario para el pulso EN del LCD.
tTimer40uS	Retardo mínimo entre comandos LCD (40 μ s).
EOB	Marcador de fin de cadena en mensajes FCC (<i>FF</i>).
Clear_LCD	Comando para limpiar el LCD.
ADD_L1	Dirección base de la línea 1 del LCD.
ADD_L2	Dirección base de la línea 2 del LCD.
PortPB	Puerto físico PH donde están conectados PB1 y PB2.
MaskPB1	Máscara del botón **PB1** , conectado al pin **PH3** .
MaskPB2	Máscara del botón **PB2** , conectado al pin **PH0** .
tSupRebPB1	Tiempo de antirrebote para PB1 (10 ms).
tShortP1	Tiempo mínimo para Short Press en PB1 (25 ms).
tLongP1	Tiempo mínimo para Long Press en PB1 (3 s).
tSupRebPB2	Tiempo de antirrebote para PB2 (10 ms).
tShortP2	Tiempo mínimo para Short Press en PB2 (25 ms).
tLongP2	Tiempo para Long Press en PB2 (3 s).
LDConfig	LED indicador del modo Configurar.
MinNumVueltas	Valor mínimo permitido para vueltas programadas (3).
MaxNumVueltas	Valor máximo de vueltas permitidas (20).
LDEspera	LED indicador del modo Espera.
LDCorrer	LED indicador del modo Correr.
tTimerCal	Tiempo usado en el cálculo de velocidad (100 ms).
tTimerError	Tiempo de despliegue para mensaje de error (2 s).
DeltaS	Valor auxiliar para cálculo de estabilidad.
DeltaM	Valor auxiliar para cálculo de promedio.
DeltaP	Valor auxiliar para cálculo de variaciones.
PortRele	Puerto donde está conectado el relé de señalización.
Rele	Máscara para activar el relé (04).
VMin	Velocidad mínima permitida (35).
VMax	Velocidad máxima permitida (90).
LDResumen	LED indicador del modo Resumen.
tTimerBrillo	Tiempo para muestreo del ADC de brillo (4×100 ms).
MaskSCF	Máscara asociada al flag del convertidor ADC.

3. Conclusiones

El proyecto permitió integrar correctamente varios periféricos del microcontrolador, como el teclado matricial, el display de 7 segmentos, el LCD, los botones y el control de brillo mediante conversión analógica digital. La interacción entre estos módulos se logró gracias al uso de máquinas de estado bien definidas y temporizadores que garantizaron estabilidad y sincronización en cada una de las tareas.

Durante el desarrollo, una de mis mayores dificultades fue la transición entre modos, especialmente en el LCD. Los mensajes no se desplegaban correctamente debido a que no se estaba realizando el borrado adecuado de la bandera LCD_OK, lo cual impedía el despliegue de los mensajes correctos. Una vez corregido esto, la comunicación con la pantalla se estabilizó y los cambios de modo ocurrieron de forma consistente.

Otra parte retadora fue la implementación del estado 5 de la tarea Correr, que concentraba gran parte de la lógica del ciclo de medición, validación, actualización de valores y despliegue. Debido a su extensión, era necesario comprender con claridad la función de cada bloque para asegurar que la máquina de estados progresara sin errores.

La medición del tiempo entre sensores simulados permitió obtener velocidades coherentes con el comportamiento esperado. A su vez, el manejo de los modos Espera, Configurar, Correr y Resumen mostró que es posible estructurar un sistema completo mediante ensamblador, administrando eventos y tiempos de forma precisa. La estructura en diagramas de cada tarea facilitó la depuración y el entendimiento del funcionamiento.

4. Recomendaciones

Recomiendo garantizar siempre que la bandera LCD_OK esté en estado válido antes de enviar un comando o carácter al LCD, ya que en el caso que no se limpie puede impedir que los mensajes cambien correctamente entre modos y ocasionar fallos de despliegue.

También es importante asegurarse del buen funcionamiento de la subrutina Calcula para evitar que errores propios de esta rutina se mezclen con fallos del estado 5 de la tarea Correr. Dado que este estado concentra gran parte de la lógica del sistema, cualquier comportamiento anómalo puede confundirse con problemas en módulos nuevos. Verificar primero la correcta operación de Calcula permite aislar fallas, mantener claridad en la depuración y continuar la ampliación del sistema de forma ordenada.

Finalmente, se recomienda depurar cada tarea de forma aislada antes de integrarla en el sistema completo. En sistemas basados en máquinas de estado y múltiples tareas concurrentes, pequeños errores pueden propagarse rápidamente y manifestarse en módulos que inicialmente no parecen relacionados. Probar cada tarea por separado permite entender su comportamiento individual, identificar condiciones que no se cumplen y verificar que las transiciones entre estados se realizan correctamente. Una vez que cada módulo demuestra estabilidad por sí mismo, la integración resulta más sencilla y cualquier fallo posterior puede localizarse con mayor facilidad, con esto la importancia de una tarea 5 correcta. Este enfoque no solo mejora la calidad del desarrollo, sino que también reduce significativamente el tiempo de depuración.

5. Bibliografía

Delgado, Geovanny. Material del curso IE623. Escuela de Ingeniería Eléctrica. UCR. 2021.